GIẢI QUYẾT VẤN ĐỀ BẰNG TÌM KIẾM

1. Mục tiêu

- Hiểu cách mô hình hóa bài toán tìm kiếm bằng không gian trạng thái, trạng thái đầu, trạng thái đích, hành động, chi phí đường đi và cây tìm kiếm.
- Cài đặt và khảo sát các chiến lược tìm kiếm không thông tin: BFS, DFS, Depth-Limited DFS và Iterative Deepening Search (IDS).
- Cài đặt và khảo sát các chiến lược tìm kiếm có thông tin: Greedy Best-First Search (GBFS), A* và Weighted A*.
- Hiểu vai trò của heuristic Manhattan/Euclidean và ảnh hưởng của kiểm tra chu trình (cycle checking).
- Đánh giá thuật toán theo độ dài đường đi, số nút mở rộng, độ sâu, kích thước frontier và thời gian chạy.

2. Dữ liệu và chương trình sử dụng

Nhóm	Tập tin	Nội dung
Cấu trúc cây	01_trees.ipynb	Lớp Node, parent, action, cost và truy hồi đường đi từ nút gốc.
So sánh thuật toán	02_Maze_Example.ipynb	BFS, DFS, DLS, IDS, GBFS, A*, Weighted A* và so sánh thời gian.
Minh họa	03_Maze_Example_Animation.ipynb	Quan sát từng bước BFS, DFS, GBFS, A* bằng animation.
BFS và A*	04_Maze_BFS_vs_A_Star.ipynb	So sánh BFS với A* dùng Euclidean và Manhattan.
Chu trình	05_Maze_Example-Animation_Cycle_checking.ipynb	Ba chiến lược cycle checking cho DFS.
Heuristic	06_Explore_heuristics.ipynb	Khảo sát giá trị heuristic trong mê cung rỗng.
Bài tập	Ex1_Maze.ipynb	Bài thực hành đầy đủ: BFS, DFS, GBFS, A*, IDS và đánh giá.
Bài tập rút gọn	Ex2_Maze_simplified.ipynb	Phiên bản tập trung vào BFS.
Quan sát dữ liệu	Ex3_Show_all_mazes.ipynb	Hiển thị toàn bộ các mê cung dùng trong bài tập.

3. Bài tập thực hành

Bài 1. Cấu trúc cây tìm kiếm và lớp Node

- Mở file 01_trees.ipynb và chạy tuần tự các cell.
- Quan sát bốn thuộc tính của mỗi nút: pos (trạng thái), parent (nút cha), action (hành động), cost (chi phí tích lũy).
- Giải thích vì sao cây tìm kiếm AI thường được tạo động và không nhất thiết là cây nhị phân.
- Dùng get_path_from_root() để truy hồi dãy trạng thái/hành động từ nút hiện tại về nút gốc.

```
class Node:
    def __init__(self, pos, parent, action, cost):
        self.pos = pos
        self.parent = parent
        self.action = action
        self.cost = cost
```

Bài 2. Mô hình hóa bài toán mê cung

- Mở Ex3_Show_all_mazes.ipynb để quan sát các mê cung: small, medium, large, open, L, loops, empty và empty_maze_2.

© DNTAI

- Xác định năm thành phần của bài toán tìm kiếm: trạng thái đầu, tập hành động, mô hình chuyển trạng thái, trạng thái đích và chi phí đường đi.
- Trong các file *.txt: X là tường; S là điểm bắt đầu; G là đích; khoảng trắng là ô có thể di chuyển.

```
Ví dụ small_maze.txt:
XXXXXXXXXXXXXXXXXXXXX
X XX        X X      X
X    XXXXXX X XXXXXX X
XXXXXX     S  X      X
...
XG         XX        X
XXXXXXXXXXXXXXXXXXXXX
```

- Tính/ước lượng các đại lượng n, d, m, b: kích thước không gian trạng thái, độ sâu lời giải tối ưu, độ sâu tối đa và hệ số phân nhánh tối đa.

Bài 3. Tìm kiếm không thông tin: BFS và DFS

- Mở 02_Maze_Example.ipynb và 03_Maze_Example_Animation.ipynb.
- Chạy BFS. Theo dõi frontier, số trạng thái được mở rộng và đường đi cuối cùng.
- Chạy DFS nhiều lần. Notebook sử dụng thứ tự hướng đi có yếu tố ngẫu nhiên nên kết quả DFS có thể thay đổi giữa các lần chạy.
- Giải thích sự khác nhau giữa Queue của BFS và Stack của DFS; liên hệ với tính đầy đủ, tối ưu và nhu cầu bộ nhớ.

Tiêu chí	BFS	DFS
Cấu trúc frontier	Queue - FIFO	Stack - LIFO
Tối ưu với chi phí bước bằng nhau	Có	Không bảo đảm
Đầy đủ trong không gian hữu hạn	Có	Có thể gặp vấn đề với nhánh sâu/chu trình
Bộ nhớ	Có thể lớn	Thường thấp hơn BFS

Bài 4. Depth-Limited DFS và Iterative Deepening Search (IDS)

- Trong 02_Maze_Example.ipynb, khảo sát Depth-Limited DFS bằng cách đặt giới hạn độ sâu.
- Chạy IDS và ghi lại số lần tăng giới hạn độ sâu trước khi tìm thấy lời giải.
- Giải thích vì sao IDS kết hợp đặc tính bộ nhớ của DFS với khả năng tìm lời giải theo độ sâu tăng dần.
- Lưu ý từ notebook: IDS phụ thuộc vào cơ chế kiểm tra chu trình của DFS; vì vậy cycle checking không phù hợp vẫn có thể ảnh hưởng kết quả.

Bài 5. Tìm kiếm có thông tin: GBFS, A* và Weighted A*

- Chạy GBFS, A* và Weighted A* trong 02_Maze_Example.ipynb.
- Mở 04_Maze_BFS_vs_A_Star.ipynb để so sánh trực tiếp BFS và A*.
- Với A*, khảo sát hai heuristic: khoảng cách Euclidean và Manhattan.

```
Manhattan: h(n) = |x_n - x_g| + |y_n - y_g|
Euclidean: h(n) = sqrt((x_n-x_g)^2 + (y_n-y_g)^2)
A*: f(n) = g(n) + h(n)
Weighted A*: f(n) = g(n) + w*h(n)
```

- Ghi nhận nhận xét của notebook: BFS và A* đều tìm đường ngắn nhất trong thiết lập này; A* có thể mở rộng ít trạng thái hơn đáng kể nếu heuristic phù hợp.

© DNTAI

- Thay đổi trọng số w của Weighted A* và quan sát đánh đổi giữa tốc độ tìm kiếm với chất lượng lời giải.

Bài 6. Ảnh hưởng của heuristic

- Mở 06_Explore_heuristics.ipynb.
- Quan sát bản đồ giá trị h(n) và f(n) trong mê cung rỗng.
- So sánh Manhattan và Euclidean cho GBFS và A*.
- Giải thích vì sao Manhattan thường phù hợp với bài toán di chuyển 4 hướng trên lưới hơn Euclidean.
- Đề xuất một heuristic khác và nêu điều kiện để heuristic dùng trong A* vẫn bảo đảm tính tối ưu.

Bài 7. Kiểm tra chu trình trong DFS

- Mở 05_Maze_Example-Animation_Cycle_checking.ipynb trên open_maze.
- Quan sát ba phương án cycle checking được minh họa trong notebook:
  - Kiểm tra trạng thái có xuất hiện trên đường đi hiện tại hay không.
  - Kiểm tra đường đi và không thêm lại trạng thái đang có trong frontier.
  - Kiểm tra đường đi và điều chỉnh frontier bằng cách đưa trạng thái tìm thấy lên đầu stack.
- So sánh số lần lặp, kích thước frontier và hiện tượng lặp trạng thái giữa ba phương án.

Bài 8. Bài tập tổng hợp - Goal-based Planning Agent

- Sử dụng Ex1_Maze.ipynb làm bài thực hành chính. Ex2_Maze_simplified.ipynb có thể dùng làm phiên bản rút gọn chỉ tập trung vào BFS.
- Hoàn thiện tác tử lập kế hoạch và các thuật toán BFS, DFS, GBFS, A*. Với phần nâng cao, thực hiện IDS và bài toán nhiều đích.
- Không lưu thông tin tìm kiếm trực tiếp vào bản đồ mê cung; tổ chức thông tin theo cây tìm kiếm/frontier/reached như notebook yêu cầu.
- Với mỗi thuật toán và mỗi mê cung, thu thập: lời giải/chi phí, số nút mở rộng, độ sâu lớn nhất và kích thước frontier lớn nhất.

4. Hướng dẫn thực hiện

4.1 Quy trình chạy notebook

1. Giải nén toàn bộ thư mục lab02 - Searching vào cùng một thư mục làm việc.

2. Mở Jupyter Notebook/JupyterLab hoặc Google Colab. Nếu dùng Colab, tải maze_helper.py và các file *.txt vào môi trường chạy hoặc mount Google Drive.

3. Chạy các cell theo đúng thứ tự từ trên xuống. Khi thay đổi thuật toán hoặc heuristic, khởi động lại kernel nếu cần để tránh dùng biến từ lần chạy trước.

4. Dùng các hàm hiển thị mê cung/animation có sẵn để kiểm tra trực quan đường đi, frontier và trạng thái đã mở rộng.

5. Không xóa các cell mô tả yêu cầu; bổ sung câu trả lời bằng Markdown và code ngay dưới từng Task.

4.2 Bảng ghi kết quả thí nghiệm

Maze	Thuật toán	Path cost	Nút mở rộng	Max depth	Max frontier	Thời gian
small	BFS					

© DNTAI

small	DFS					
small	GBFS					
small	A*					
medium	BFS					
medium	A*					
large	BFS/A*					
open	DFS + cycle check					

Có thể bổ sung các dòng cho L_maze, loops_maze, empty_maze và empty_maze_2.

4.3 Câu hỏi thảo luận

6. Khi nào BFS dùng nhiều bộ nhớ hơn A*? Vì sao heuristic tốt làm giảm vùng trạng thái phải khảo sát?
7. Tại sao DFS có thể cho đường đi rất dài hoặc gặp vấn đề trong môi trường có chu trình?
8. Tại sao thứ tự sinh các trạng thái con ảnh hưởng mạnh đến DFS nhưng ít ảnh hưởng hơn đến BFS?
9. So sánh GBFS và A*: thuật toán nào sử dụng g(n), h(n), f(n)? Ảnh hưởng đến tính tối ưu như thế nào?
10. Manhattan và Euclidean khác nhau ra sao trong lưới chỉ cho phép đi lên/xuống/trái/phải?
11. Weighted A* thay đổi điều gì khi tăng trọng số heuristic?
12. Cycle checking làm giảm lặp trạng thái nhưng có thể làm tăng chi phí quản lý dữ liệu như thế nào?

5. Yêu cầu sản phẩm nộp

- Notebook Ex1_Maze.ipynb, Ex2_Maze_simplified.ipynb, và Ex3_Show_all_mazes.ipynb đã hoàn thiện.
- Bảng kết quả so sánh ít nhất BFS, DFS, GBFS và A* trên tối thiểu 3 mê cung.
- Ít nhất 2 hình/animation hoặc ảnh chụp kết quả minh họa đường đi và vùng trạng thái được tìm kiếm.
- Phần nhận xét 1-2 trang về tính đầy đủ, tối ưu, thời gian, bộ nhớ, heuristic và cycle checking.
- Mã nguồn phải chạy lại được từ đầu; không để lỗi, output debug dư thừa hoặc đường dẫn tuyệt đối phụ thuộc máy cá nhân.
- Báo cáo có trang bìa, mục lục và nội dung bài làm ghi rõ họ tê, mã sinh viên của từng thành viên nhóm.

Tóm tắt kiến thức cần nhớ

Thuật toán	Hàm ưu tiên / frontier	Đặc điểm chính
BFS	FIFO / độ sâu nhỏ trước	Đầy đủ; tối ưu khi chi phí bước bằng nhau; có thể tốn bộ nhớ.
DFS	LIFO / đi sâu trước	Bộ nhớ thấp hơn; không bảo đảm tối ưu; cần xử lý chu trình.
IDS	DFS với giới hạn tăng dần	Tìm theo độ sâu; dung hòa ưu điểm BFS/DFS trong nhiều trường hợp.
GBFS	h(n)	Tập trung về phía đích; nhanh nhưng không bảo đảm tối ưu.
A*	f(n)=g(n)+h(n)	Tối ưu với heuristic phù hợp; hiệu quả phụ thuộc chất lượng heuristic.
Weighted A*	g(n)+w·h(n)	Tăng định hướng heuristic, đổi lấy khả năng mất tối ưu khi w>1.

© DNTAI